

# Curso de Gestión y Tecnologías de Procesos de Negocio

## Práctica: Implementación y APIs de Flowable.

### Objetivos

El objetivo de esta práctica es profundizar en aspectos de implementación y ejecución de procesos de negocio utilizando *Flowable 6.8* como motor BPMN.

Hasta el momento se ha trabajado principalmente con el modelado y la ejecución de procesos mediante las herramientas provistas por Flowable. En esta práctica se abordarán mecanismos que permiten extender el comportamiento de los procesos e interactuar con el motor de ejecución mediante código.

En particular, se trabajará con:

- *Execution Listeners* y *Task Listeners*, para ejecutar código asociado a eventos que ocurren durante la ejecución de procesos y tareas;
- la API Java de Flowable, para desplegar procesos, iniciar instancias, consultar y completar tareas, administrar variables y obtener información sobre la ejecución y el historial de los procesos;
- la API REST de Flowable, como mecanismo alternativo para interactuar con el motor desde aplicaciones externas;

El propósito es comprender cómo una aplicación puede interactuar con un motor BPMN y cómo el comportamiento definido gráficamente en un modelo de proceso puede complementarse mediante código y servicios externos.

### Ejercicios

#### 1 Listeners

- a) Experimentar con los eventos asociados a los *Execution Listeners* y *Task Listeners* de Flowable. Diseñe un proceso que contenga al menos una tarea de usuario y asocie listeners a diferentes eventos del proceso, los flujos de secuencia y la tarea.

Implemente los listeners mediante clases Java y utilice la información proporcionada por *DelegateExecution* y *DelegateTask* para identificar el evento que produjo cada invocación.

Ejecute el proceso y determine el orden en que se producen los distintos eventos durante su ejecución.

- b) Modificar el proceso anterior incorporando tareas *multi-instancia* (MI) y subprocessos. Asociar *Execution Listeners* y *Task Listeners* a estos elementos y analizar cómo se comportan los distintos eventos durante su ejecución.

En particular, observar cuántas veces se ejecuta cada listener y en qué orden se producen los eventos para las distintas instancias de una tarea MI y para las actividades contenidas en un subprocesso.

- c) Experimentar con tareas MI basadas en una colección (*Multiple Instances without a Priori Run-Time Knowledge*). Crear la colección desde código Java al iniciar la instancia del proceso y analizar, durante la ejecución, las variables asociadas a cada instancia de la tarea.

Utilizar *Task Listeners* para acceder al elemento de la colección correspondiente a cada instancia y mostrar su valor.

---

## 2 Unit Testing

Flowable permite realizar pruebas unitarias de los procesos BPMN utilizando JUnit. Estas pruebas permiten ejecutar un proceso desde código Java y verificar automáticamente, mediante aserciones, que durante su ejecución se alcanzan los estados esperados.

En el repositorio de la asignatura se proporciona un ejemplo de una clase JUnit que utiliza *FlowableTest*, junto con los archivos de configuración necesarios. También se indica la dependencia de JUnit que debe agregarse al archivo *pom.xml* del proyecto.

Para realizar el ejercicio tener en cuenta las siguientes cuestiones:

- Las clases correspondientes a las pruebas deben ubicarse en *src/test/java*. Se recomienda utilizar el mismo paquete empleado por las clases del proyecto.
  - El archivo *flowable.cfg.xml* debe ubicarse en *src/test/resources*. La configuración proporcionada utiliza una base de datos H2 en memoria, por lo que los tests se ejecutan sobre un motor Flowable independiente.
  - El modelo BPMN utilizado por el test debe estar disponible como un recurso del proyecto. Mediante la anotación *@Deployment* puede indicarse el archivo BPMN que debe ser desplegado automáticamente antes de ejecutar el test.
  - La instancia del proceso puede iniciarse mediante *RuntimeService*, mientras que *TaskService* permite consultar y completar las tareas de usuario.
  - Las pruebas deben utilizar aserciones de JUnit para verificar los estados alcanzados durante la ejecución del proceso.
- a) Diseñar en Flowable Modeler un proceso que contenga al menos una tarea de usuario con un formulario asociado (utilizando la propiedad *Form properties*). El formulario deberá contener campos cuyos valores sean utilizados posteriormente por el proceso para determinar el flujo de ejecución.
- Por ejemplo, el formulario puede permitir ingresar los datos de una solicitud y seleccionar si ésta debe ser aprobada o rechazada. Utilizar posteriormente un gateway exclusivo para seleccionar diferentes caminos de acuerdo con los valores ingresados.
- b) A partir del ejemplo JUnit proporcionado, implementar una unidad de testeo que despliegue el proceso, inicie una instancia y verifique que ésta alcanza la tarea que contiene el formulario.
- c) Desde la unidad de testeo, completar la tarea proporcionando mediante la API Java los valores correspondientes a los campos del formulario. Tener en cuenta que desde el test no se completa visualmente el formulario: los valores deben proporcionarse mediante variables al completar la tarea.
- d) Verificar mediante aserciones que, de acuerdo con los valores proporcionados, el proceso continúa por el camino esperado y alcanza la tarea correspondiente.
- e) Implementar al menos dos casos de prueba utilizando diferentes valores para los campos del formulario, de manera que el proceso siga caminos diferentes.

## 3 Interacción con la API del Motor de Flowable

En los siguientes ejercicios se utilizará la API Java de Flowable para interactuar con el motor. Los programas Java deberán conectarse a la misma base de datos PostgreSQL utilizada por el servidor Flowable, permitiendo consultar y modificar el estado de las instancias de procesos ejecutadas en el servidor.

En el repositorio de la asignatura se proporcionarán ejemplos de código Java para utilizar como punto de partida en la resolución de los siguientes ejercicios.

a) Utilizando *TaskService*:

- a) Lanzar una o más instancias de procesos en las cuales un usuario tenga tareas asignadas. Mediante la API Java de Flowable, consultar la lista de tareas asignadas a dicho usuario.

- 
- b) Consultar la lista de tareas disponibles para los usuarios (candidatos) pertenecientes al grupo *Alumnos*.
  - c) Completar mediante la API Java una tarea asignada a un usuario.
  - d) Utilizando el proceso *Proceso Con Formulario*, completar mediante la API Java la tarea *Solicitar Datos*, proporcionando valores para los campos del formulario. Verificar posteriormente qué tarea queda en espera de acuerdo con los valores proporcionados.
  - e) Listar las tareas en espera correspondientes a todas las instancias activas de un proceso determinado. Para cada tarea mostrar el identificador de la instancia, el nombre de la tarea y el usuario al que se encuentra asignada.
  - f) Para una tarea en espera, consultar mediante *TaskService* las variables asociadas a la misma. En el caso del proceso *Proceso Con Formulario*, mostrar los valores de las variables *observaciones* y *decision* luego de completar la tarea *Solicitar Datos*.
  - g) Realizar diferentes consultas sobre las tareas en espera, combinando criterios tales como usuario asignado, grupo candidato, nombre de la tarea y definición del proceso. Mostrar además la cantidad de tareas que satisface cada consulta.
- b) Utilizando *RuntimeService*:
- a) Consultar todas las instancias activas del proceso *Proceso Con Formulario*. Para cada instancia mostrar su identificador, el identificador de la definición del proceso y las variables actualmente almacenadas en la instancia.  
Mostrar además la cantidad total de instancias activas del proceso.
  - b) Seleccionar una instancia activa del proceso *Proceso Con Formulario* y consultar las variables asociadas a la misma.  
Establecer (crear si no existe) o modificar el valor de una variable mediante la API Java y consultar nuevamente las variables de la instancia para verificar el cambio realizado.
- c) Utilizando *HistoryService*:
- a) Listar todos los procesos finalizados.
  - b) Dado un proceso, listar la duración de todas las instancias finalizadas.
  - c) Listar los tiempos de una actividad determinada. Mostrar: cuándo comenzó, cuándo finalizó, su duración, a quién fue asignada y a qué definición de proceso pertenece.
  - d) Para una instancia de proceso finalizada, reconstruir el recorrido realizado por la instancia, listando las actividades ejecutadas en orden cronológico.
  - e) Para una instancia de proceso finalizada, listar las variables utilizadas durante su ejecución, indicando su nombre y valor.
  - f) Dado un usuario, listar las tareas que completó, indicando el proceso al que pertenecen, la fecha de inicio, la fecha de finalización y su duración.
  - g) Para una definición de proceso, calcular el tiempo promedio de ejecución de sus instancias finalizadas e identificar las instancias de menor y mayor duración.

## 4 Interacción con la API REST de Flowable

Los ejercicios anteriores permiten interactuar con el motor de Flowable mediante su API Java. Flowable proporciona además una API REST que permite realizar operaciones equivalentes utilizando peticiones HTTP.

En esta sección se deberán resolver, utilizando la API REST de Flowable, los mismos ejercicios planteados anteriormente para *TaskService*, *RuntimeService* e *HistoryService*.

Para realizar y analizar las peticiones HTTP se utilizará inicialmente **Postman**. En cada ejercicio se deberá identificar el recurso REST correspondiente, el método HTTP utilizado (GET, POST, PUT, etc.), los parámetros necesarios y, cuando corresponda, el contenido enviado en el cuerpo de la petición.

Configurar en Postman una variable `baseUrl` con la URL base de la API REST:

---

`http://localhost:8080/flowable-ui/process-api`

De esta forma, las peticiones podrán escribirse, por ejemplo, como:

```
{{baseUrl}}/runtime/process-instances  
{{baseUrl}}/history/historic-process-instances
```

Utilizar autenticación *Basic Auth* con un usuario válido de Flowable.

Para cada uno de los ejercicios de `TaskService`, `RuntimeService` e `HistoryService` planteados en la sección anterior:

1. Cuando la operación requiera enviar información al motor, configurar el cuerpo de la petición en Postman utilizando `Body -> raw -> JSON`.
2. Analizar la respuesta JSON obtenida e identificar los principales datos retornados por el motor.

En particular, prestar atención a la diferencia entre los identificadores de los elementos definidos en el modelo BPMN y los identificadores generados por Flowable para las instancias de procesos, ejecuciones y tareas.

## 5 Interacción con la API REST desde código

En esta sección se utilizará la API REST de Flowable desde un programa. El lenguaje de programación y las bibliotecas utilizadas quedan a elección del alumno.

El programa deberá realizar las peticiones HTTP correspondientes a la API REST de Flowable y procesar las respuestas JSON obtenidas. No se deberá utilizar directamente la API Java de Flowable para resolver estos ejercicios.

Se recomienda reutilizar las consultas realizadas previamente mediante Postman.

Resolver los siguientes ejercicios:

1. Consultar las tareas actualmente asignadas a un usuario determinado. Mostrar para cada tarea su identificador, nombre, usuario asignado e identificador de la instancia de proceso a la que pertenece.
2. Dado el identificador de una instancia de proceso activa, consultar sus variables y mostrar el nombre y valor de cada una de ellas.
3. Consultar todas las instancias finalizadas de un proceso determinado. Ordenarlas de mayor a menor según su duración y mostrar para cada una su identificador y duración.
4. A partir de las instancias finalizadas de un proceso, calcular la duración promedio, mínima y máxima de las mismas.
5. Dada una instancia de proceso finalizada, consultar sus actividades históricas y reconstruir el recorrido realizado por la instancia, mostrando las actividades ordenadas cronológicamente según su instante de inicio. Para cada actividad mostrar su nombre, instante de inicio, instante de finalización y duración.

---

## A Creación de un proyecto Maven para Flowable en Eclipse

Para desarrollar los ejemplos de esta práctica se utilizará un proyecto Maven en Eclipse con Flowable 6.8.0 y Java 11.

Para crear el proyecto realizar los siguientes pasos:

1. En Eclipse seleccionar:

`File -> New -> Other...`

`Maven -> Maven Project`

y presionar *Next*.

2. Seleccionar la opción:

`Create a simple project (skip archetype selection)`

3. Completar los datos del proyecto. Por ejemplo:

```
Group Id:      curso.flowable
Artifact Id:   CursoFlowable
Version:       0.0.1-SNAPSHOT
Packaging:     jar
```

Finalmente presionar *Finish*.

4. Abrir el archivo `pom.xml` generado por Maven y configurar Java 11 e incorporar las dependencias necesarias para utilizar Flowable.

Ver o descargar el archivo desde el aula virtual del curso.

La base de datos H2 será utilizada para los ejemplos que ejecutan un motor Flowable directamente desde Java. El driver PostgreSQL será utilizado para los ejemplos que acceden a la base de datos persistente utilizada por Flowable ejecutado mediante Docker.

5. Guardar el archivo `pom.xml`. Maven descargará las dependencias necesarias.

Si Eclipse no actualiza automáticamente las dependencias, seleccionar:

Botón derecho sobre el proyecto

-> `Maven`

-> `Update Project...`

-> `OK`

6. Verificar en el *Package Explorer* que el proyecto contenga una estructura similar a la siguiente:

```
CursoFlowable
|
+-- src/main/java
|
+-- src/main/resources
|
+-- src/test/java
|
+-- Maven Dependencies
|
+-- pom.xml
```

Dentro de *Maven Dependencies* deberá aparecer la biblioteca `flowable-engine-6.8.0.jar` junto con las demás dependencias incorporadas por Maven.

7. Para crear un paquete Java, seleccionar:

```
src/main/java
-> botón derecho
-> New
-> Package
```

Por ejemplo, utilizar como nombre:

```
curso.flowable
```

8. Los archivos que contienen las definiciones de procesos BPMN pueden almacenarse dentro de:

```
src/main/resources/processes
```

Por ejemplo:

```
CursoFlowable
|
+-- src/main/java
|   |
|   +-- curso.flowable
|       |
|       +-- PrimerEjemplo.java
|
+-- src/main/resources
|   |
|   +-- processes
|       |
|       +-- miprimerproceso.bpmn20.xml
|
+-- src/test/java
|
+-- Maven Dependencies
|
+-- pom.xml
```

De esta forma, el mismo proyecto podrá utilizarse tanto para ejecutar un motor Flowable directamente desde programas Java como para realizar ejercicios utilizando la API Java sobre la base de datos PostgreSQL persistente utilizada por Flowable.

## References

- [1] Flowable 6.8 Java API (Javadoc). <https://javadoc.io/doc/org.flowable/flowable-engine/6.8.0/index.html>
- [2] Flowable REST API. <https://www.flowable.com/open-source/docs/bpmn/ch14-REST>
- [3] Postman – API Testing. <https://www.postman.com>
- [4] Jersey – RESTful Web Services in Java. <https://eclipse-ee4j.github.io/jersey/>